

PYTHON BÁSICO · PROGRAMACIÓN DESDE CERO

Módulos 1 & 2

Introducción a Python · Variables y Tipos de Datos

● Módulo 1 · Sesión 1 · 3h

● Módulo 2 · Sesión 2 · 3h

Lic. Hebert Erland Cussi Cuentas · 19:00 – 22:00 · Nivel básico, sin requisitos previos

MÓDULO 1

Introducción a Python

Sesión 1 · 3 horas — objetivo: instalar Python y VS Code, ejecutar el primer programa y entender para qué sirve Python hoy.

De un proyecto navideño al dominio global

1989 - 1991 · El origen

Creado por **Guido van Rossum** en Países Bajos, como sucesor del lenguaje ABC.

El nombre no viene de la serpiente: es un homenaje a los comediantes **Monty Python**.

2000 - 2008 · La evolución

Python 2.0 (2000): recolección de basura y soporte Unicode.

Python 3.0 (2008): versión rompedora que limpió la sintaxis. La transición duró más de una década.

Hoy · Dominio del mercado

Lenguaje líder en **IA, Data Science y Automatización**.

Ocupa el **puesto #1** del Índice TIOBE, por encima de C, Java y C++.

¿Qué es Python?

Lenguaje de **alto nivel**, interpretado y de propósito general, creado por **Guido van Rossum en 1991**.

Su sintaxis está muy cerca del lenguaje humano: el programador no gestiona memoria ni detalles del procesador a mano.



Datos & IA

NumPy, Pandas, TensorFlow



Web

Django, FastAPI, Flask



Automatización

Scripts, bots, tareas repetitivas



Ciberseguridad

Pentesting, hacking ético



Finanzas

Análisis cuantitativo, trading



Índice TIOBE

Python: puesto #1 en popularidad

Instalación del entorno

1 Instalar Python 3

python.org → Downloads → Python 3.12+

Windows: marcar ☒ 'Add Python to PATH'

```
python --version  
# Python 3.12.x
```

2 Instalar VS Code

code.visualstudio.com → descargar

Instalar extensiones: **Python** (Microsoft) y **Pylance** para autocompletado.

¡Hola, Mundo! – Tu primer programa

1. Crear carpeta `python-basico/`
2. VS Code → Open Folder
3. Crear archivo `hola.py`
4. Escribir el código →
5. Ejecutar: `python hola.py`

```
# hola.py
print("¡Hola, mundo!")
print("Bienvenido a Python")
print("Sesión 1 – ¡Empezamos!")
```

 SALIDA EN PANTALLA

```
¡Hola, mundo!
Bienvenido a Python
Sesión 1 – ¡Empezamos!
```

Comandos esenciales de la terminal

`cd carpeta`

Entrar a una carpeta

`cd ..`

Regresar a la carpeta anterior

`ls / dir`

Listar archivos

`python archivo.py`

Ejecutar un script

`python`

Abrir intérprete interactivo

`exit()`

Salir del intérprete

`Ctrl + C`

Detener programa en ejecución

Comentarios en el código

Los comentarios son líneas que Python ignora al ejecutar. Sirven para documentar y explicar el código.

```
# Esto es un comentario de una línea  
print("Hola") # comentario al final
```

```
"""  
Esto es un comentario de  
múltiples líneas – también llamado docstring  
"""
```

💡 Un buen programador escribe comentarios para que otro (o el yo del futuro) entienda el código sin leer cada línea.

Errores comunes — no les temas

Los errores son parte del aprendizaje. Python te dice exactamente qué falló.

SyntaxError

Falta comilla, paréntesis o palabra mal escrita.

```
print('Hola' # falta )
```

IndentationError

Sangría incorrecta: Python es estricto con los espacios.

```
print('mal')
```

NameError

Variable o función que no existe o está mal escrita.

```
prnit('Hola') # typo
```

TypeError

Operación entre tipos incompatibles.

```
"25" + 5
```

Lo que hace único a Python

Interpretado

No hay que compilar: como un intérprete de idiomas en vivo, traduce mientras hablas. Ciclo de prueba-error corto.

Tipado dinámico y fuerte

No declaras el tipo — Python lo deduce. Pero no permite sumar número y texto sin más: evita errores silenciosos.

"Baterías incluidas"

Librería estándar inmensa: correos, archivos, bases de datos, internet — sin instalar nada extra.

Multiparadigma

Imperativo, funcional (lambda, map, filter) y orientado a objetos — según lo que necesite el proyecto.

 **pip** + **PyPI**: cientos de miles de librerías gratuitas a un `pip install pandas` de distancia.

PEP 20 • THE ZEN OF PYTHON

El Zen de Python

Bello es mejor que feo.

Explícito es mejor que implícito.

Simple es mejor que complejo.

La legibilidad cuenta.

 Dato curioso: Guido van Rossum fue conocido como el **BDFL** (Benevolent Dictator For Life), hasta que renunció a ese título en 2018 para dejar el liderazgo en manos de un consejo directivo.

MÓDULO 2

Variables y Tipos de Datos

Sesión 2 · 3 horas — objetivo: declarar variables, identificar tipos fundamentales, convertir entre tipos y usar `input()`.

¿Qué es una variable?

 **Analogía:** una variable es como una caja etiquetada — tiene un nombre y guarda un valor que puedes cambiar cuando quieras.

```
nombre = 'Ana'
edad   = 25
altura = 1.65
activo = True
print(nombre) # Ana
```

Reglas para nombrar variables

- ✓ `mi_nombre` Letras, números, guión bajo
- ✓ `edad2024` Puede tener números (no al inicio)
- ✗ `2nombre` No puede empezar con número
- ✗ `mi nombre` Sin espacios
- ✗ `for` No usar palabras reservadas

Los 4 tipos de datos fundamentales

 **int**

Números enteros

```
edad = 25  
año = 2024  
temp = -10
```

 **float**

Números decimales

```
precio = 19.99  
pi = 3.14159  
desc = 0.15
```

 **str**

Cadenas de texto

```
nombre = "Ana"  
ciudad = 'La Paz'  
msg = "Hola!"
```

 **bool**

Verdadero o falso

```
activo = True  
pagado = False  
print(5>3) # True
```



Usa `type(variable)` para saber qué tipo de dato contiene — muy útil para depurar.

Conversión entre tipos de datos

`int(x)`

→ entero

```
int('25') → 25  
int(3.9) → 3
```

`float(x)`

→ decimal

```
float("3.5") → 3.5  
float(5) → 5.0
```

`str(x)`

→ texto

```
str(100) → "100"  
str(3.14) → "3.14"
```

`bool(x)`

→ booleano

```
bool(1) → True  
bool(0) → False
```

❌ Error común

```
edad = "25"  
print(edad + 5) # TypeError!
```

✅ Correcto

```
edad = "25"  
print(int(edad) + 5) # 30
```

⚠️ Regla de oro: Python **no** convierte tipos automáticamente — siempre debes hacerlo de forma explícita.

Solicitar datos al usuario – input()

input() pausa el programa y espera que el usuario escriba algo. Siempre devuelve un string.

```
# Solicitar texto
nombre = input("¿Cuál es tu nombre? ")
print("Hola, " + nombre + "!")

# Solicitar número (¡convertir!)
edad = int(input("¿Cuántos años tienes? "))
print("En 10 años tendrás", edad + 10, "años")
```



Recuerda:

input() SIEMPRE devuelve string. Si el usuario escribe '25', para Python es el texto '25', no el número 25. Usa int() o float() para convertirlo antes de operar.

Ejercicio integrador — Módulo 2

 Mini proyecto: solicitar datos al usuario y mostrar una tarjeta de presentación

```
nombre = input("Tu nombre: ")
edad = int(input("Tu edad: "))
ciudad = input("Tu ciudad: ")
anio_nac = 2024 - edad
print("-" * 28)
print(" ¡Hola,", nombre + "!")
print(" Tienes", edad, "años.")
print(" Eres de", ciudad + ".")
print(" Naciste en", anio_nac)
```

EJEMPLO DE SALIDA

```
Tu nombre: Ana
Tu edad: 22
Tu ciudad: La Paz
```

```
¡Hola, Ana!
Tienes 22 años.
Eres de La Paz.
Naciste en 2002
```

Lo que cubrimos hoy

Módulo 1

- Qué es Python y sus usos
- Instalar Python y VS Code
- Ejecutar scripts con print()
- Usar la terminal
- Escribir comentarios

Módulo 2

- Declarar variables
- Tipos: int, float, str, bool
- Función type()
- Convertir entre tipos
- Solicitar datos con input()

 Próxima sesión: **Módulo 3 – Operadores y Expresiones**